



School:Campus:

Academic Year: Subject Name: Subject Code:

Semester: Program: Branch: Specialization:

Date:

Applied and Action Learning

(Learning by Doing and Discovery)

Name of the Experiment: Build DeFi – AMM or Lending Prototype

***Coding Phase: Pseudo Code / Flow Chart / Algorithm**

1. Create two ERC-20 tokens named TokenA and TokenB.
2. Write Solidity smart contracts for TokenA and TokenB implementing the ERC-20 interface (name, symbol, decimals, total supply, transfer, approve, transferFrom).
3. Compile and deploy both token contracts to the Sepolia test network using Remix IDE.
4. Save the deployed contract addresses for TokenA and TokenB for later interaction.
5. Open MetaMask and import both tokens by pasting their contract addresses so you can view and manage balances.
6. Write an AMM (automated market maker) contract in Solidity (for example, swap.sol) to manage liquidity pools and token swaps.
7. In the AMM contract, implement core functions such as addLiquidity() and swapTokens(), and any helper functions for reserve accounting and price calculation.
8. Compile the AMM contract and deploy it to the same network where TokenA and TokenB are deployed.
9. Save the deployed AMM contract address for approvals and interactions.
10. Approve the AMM contract to spend TokenA on your behalf by calling the ERC-20 approve() function (this can be done via MetaMask or Remix).
11. Similarly, approve the AMM contract to spend TokenB.
12. Call addLiquidity() on the AMM contract to deposit amounts of TokenA and TokenB into the liquidity pool.
13. Confirm liquidity was added successfully by checking the AMM contract's reserve balances or return values from the addLiquidity() call.
14. Use the AMM's swapTokens() function to exchange TokenA for TokenB or vice versa, following the AMM's pricing formula and required parameters.
15. Confirm the swap by checking updated token balances in your wallet and the AMM contract reserves.
16. Verify all relevant transactions on the blockchain explorer for Sepolia (for example, transaction receipts and logs).
17. Re-check updated balances in MetaMask to ensure the operations (approvals, liquidity add, swap) reflected correctly.

***Software used :**

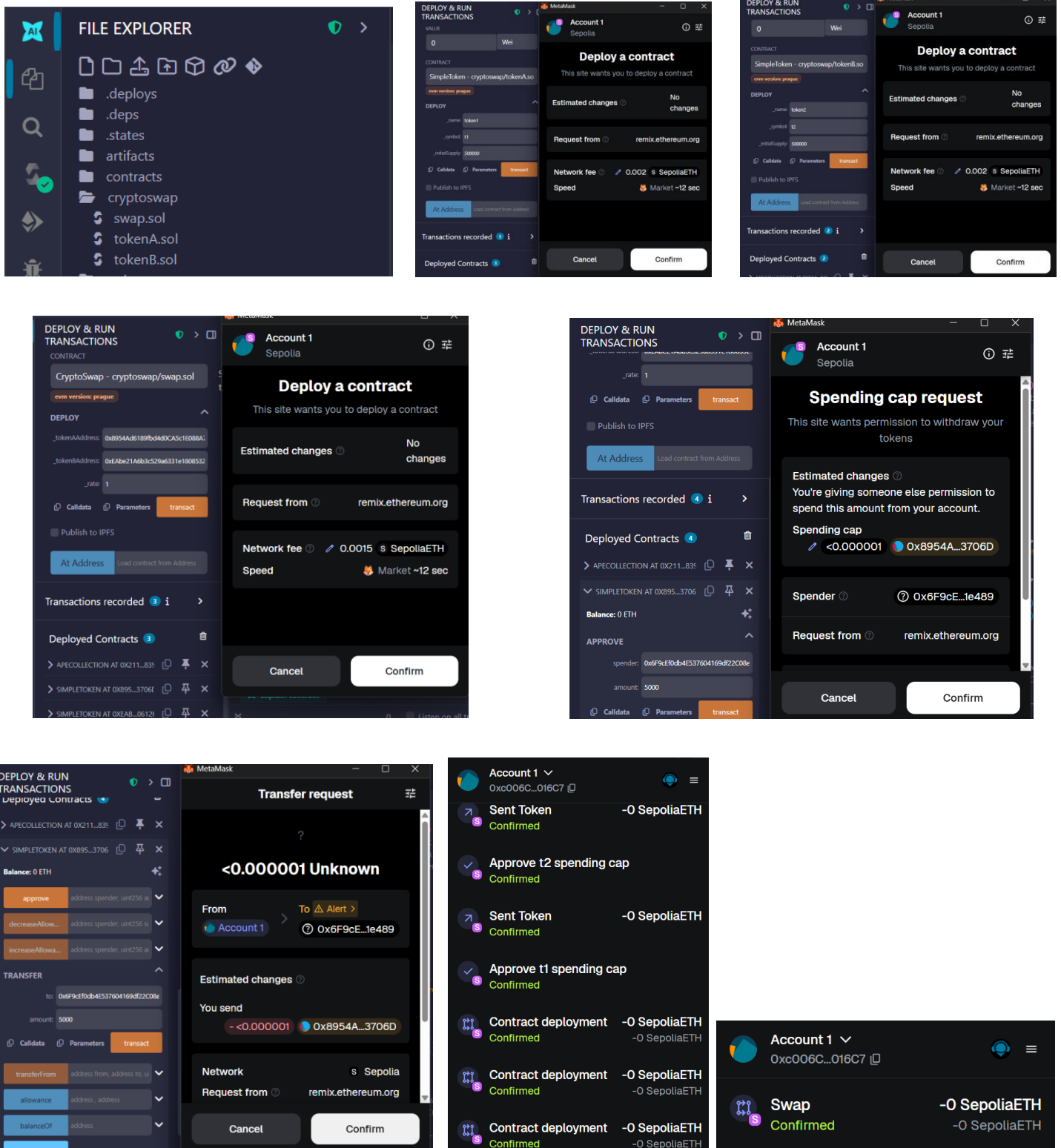
- Web browser (Brave or Microsoft edge)
- MetaMask wallet
- Ethereum Sepolia Test Network
- Remix IDE

*** Testing Phase: Compilation of Code (error detection)**

No Error

** As applicable according to the experiment.
Two sheets per experiment (10-20) to be used.*

* Implementation Phase: Final Output (no error)



*** As applicable according to the experiment.
Two sheets per experiment (10-20) to be used**

*** Observation :**

- The tokens (TokenA and TokenB) were created and appeared correctly in MetaMask.
- The AMM smart contract successfully accepted both tokens when adding liquidity.
- Swapping between the tokens worked, and balances were updated correctly.
- All actions were confirmed on the test network without any problems.

ASSESSMENT

Rubrics	Full Mark	Marks Obtained	Remarks
Concept	10		
Planning and Execution/Practical Simulation/ Programming	10		
Result and Interpretation	10		
Record of Applied and Action Learning	10		
Viva	10		
Total	50		

Signature of the Student :

Name :

Signature of the Faculty :

Regn. No. :

**** As applicable according to the experiment.
Two sheets per experiment (10-20) to be used***